

Bisimulation sémantique entre un DSL déclaratif de planification et MiniZinc

Notes de recherche – Master 2

9 mai 2026

Résumé

Ce rapport étudie la préservation sémantique entre le langage spécifique au domaine PLANNINGSPEC et le modèle MINIZINC généré automatiquement par l'implémentation du projet. Le but n'est pas de prouver le fonctionnement interne des solveurs utilisés par MINIZINC, mais de montrer que la traduction PLANNINGSPEC vers MINIZINC conserve le sens du problème de planification exprimé par l'utilisateur. Nous formulons cette étude comme une bisimulation sémantique adaptée aux langages déclaratifs : chaque solution valide du DSL doit correspondre à une affectation valide des variables MINIZINC, et toute solution retournée par MINIZINC doit pouvoir être décodée en une solution valide du DSL. Pour les préférences, la preuve porte en plus sur la conservation du score d'optimisation.

Table des matières

1	Introduction	3
2	Vue générale du pipeline	3
3	Notion de bisimulation sémantique	3
3.1	Pourquoi parler de bisimulation ?	3
4	Modèle sémantique du DSL	4
5	Modèle sémantique MiniZinc généré	5
6	Fonctions de correspondance	5
6.1	Fonction d'encodage	5
6.2	Fonction de décodage	6
7	Correspondance élément par élément	6
7.1	Temps	6
7.2	Activités	7
7.3	Ressources typées	8
7.4	Rôles	8
7.5	Affectation générale des ressources	9

8	Préservation des contraintes dures	10
8.1	Contrainte <code>mandatory_roles</code>	10
8.2	Contrainte <code>cardinality_per_activity</code>	10
8.3	Contrainte <code>resource_exclusivity</code>	11
8.4	Non-chevauchement global des ressources	12
8.5	Contrainte <code>fixed_assignment</code>	13
8.6	Contrainte <code>forbidden_assignment</code>	13
8.7	Contrainte <code>temporal_precedence</code>	14
8.8	Contrainte <code>instance_precedence</code>	14
8.9	Contrainte <code>time_window</code>	15
8.10	Contrainte <code>required_resource</code>	15
9	Préservation des préférences	15
9.1	Préférence <code>avoid_participation_on_date</code>	16
9.2	Préférence <code>preferred_resource</code>	16
9.3	Préférence <code>max_per_scope</code>	17
9.4	Préférence <code>room_stability_for_role</code>	17
9.5	Préférence <code>compact_schedule_for_role</code>	18
10	Théorème principal de bisimulation	18
11	Table de synthèse de la correspondance	19
12	Hypothèses et limites de la preuve	20
13	Conclusion	21

1 Introduction

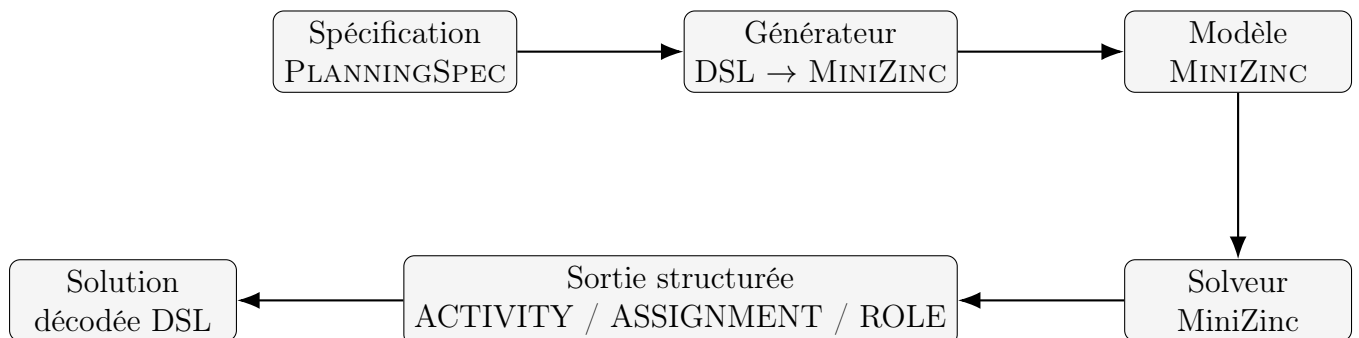
Le projet étudié propose un langage déclaratif, nommé `PLANNINGSPEC`, permettant de spécifier des problèmes de planification. Une spécification contient des jours, des créneaux, des activités, des ressources, des rôles, des contraintes dures et des préférences. La résolution effective est assurée par une génération automatique vers `MINIZINC`, puis par l'appel à un solveur compatible avec `MINIZINC`.

Cette architecture soulève une question fondamentale : le modèle `MINIZINC` généré résout-il exactement le problème exprimé dans le DSL ? Autrement dit, le générateur ne doit ni ajouter des solutions qui ne sont pas valides dans le DSL, ni supprimer des solutions qui devraient être autorisées, ni modifier le score associé aux préférences.

L'objectif de ce rapport est de fournir une étude claire de cette correspondance. On appelle ici *bisimulation sémantique* la relation bidirectionnelle entre les solutions du DSL et les solutions `MINIZINC` générées. Dans ce contexte déclaratif, la notion de bisimulation ne compare pas des transitions d'automates ; elle compare deux interprétations d'un même problème de planification.

2 Vue générale du pipeline

Le pipeline considéré est le suivant :



Dans le code du projet, les éléments essentiels de ce pipeline sont les suivants :

- la grammaire du langage est définie dans `planning-spec.langium` ;
- le générateur `MINIZINC` est défini dans `planning-spec-minizinc-generator.ts` ;
- la sortie `MINIZINC` est structurée sous les préfixes `ACTIVITY:`, `ASSIGNMENT:` et `ROLE:` ;
- le décodage de cette sortie est assuré côté serveur et côté interface par des parseurs dédiés.

3 Notion de bisimulation sémantique

3.1 Pourquoi parler de bisimulation ?

La bisimulation classique est utilisée pour comparer deux systèmes de transition : si un système effectue une action, l'autre doit pouvoir effectuer une action correspondante,

et inversement. Ici, le DSL et MINIZINC ne sont pas des systèmes de transition. Ils sont plutôt deux langages déclaratifs qui décrivent un même ensemble de solutions.

Nous utilisons donc le terme *bisimulation sémantique* dans un sens adapté : deux modèles sont bisimilaires s'ils décrivent le même espace de solutions, à encodage et décodage près.

Définition 3.1 (Bisimulation sémantique adaptée). *Soit P une spécification PLANNING-SPEC et soit $\text{Gen}(P)$ le modèle MINIZINC généré à partir de P . On dit que P et $\text{Gen}(P)$ sont sémantiquement bisimilaires si :*

- 1) *toute solution valide du DSL peut être encodée en une solution valide du modèle MINIZINC ;*
- 2) *toute solution valide du modèle MINIZINC peut être décodée en une solution valide du DSL ;*
- 3) *si des préférences existent, le score d'une solution est conservé par la traduction.*

Cette définition se décompose en trois propriétés :

Correction. MiniZinc ne doit pas produire de fausses solutions.

Complétude. MiniZinc ne doit pas perdre de solutions valides du DSL.

Préservation du score. Les préférences du DSL doivent induire la même optimisation dans MINIZINC.

4 Modèle sémantique du DSL

Une spécification PLANNINGSPEC est un objet de la forme :

```
{
  "time": ...,
  "activities": ...,
  "resources": ...,
  "roles": ...,
  "constraints": ...,
  "preferences": ...
}
```

On note une spécification DSL par :

$$P = (T, A, R, L, C, Q)$$

où :

- T est la structure temporelle ;
- A est l'ensemble des activités et de leurs instances ;
- R est l'ensemble des ressources typées ;
- L est l'ensemble des rôles applicables aux activités ;
- C est l'ensemble des contraintes dures ;
- Q est l'ensemble des préférences souples.

Définition 4.1 (Solution DSL). *Une solution DSL associe à chaque instance d'activité :*

- un créneau global de début ;
- un ensemble de ressources participantes ;
- pour chaque rôle applicable, une ou plusieurs ressources jouant ce rôle selon les contraintes de cardinalité.

Elle est valide si elle respecte toutes les contraintes dures du DSL.

5 Modèle sémantique MiniZinc généré

Le générateur produit un modèle MINIZINC construit autour de quatre familles d'objets :

- des constantes de temps : `NUM_DAYS`, `SLOTS_PER_DAY`, `TOTAL_SLOTS` ;
- des énumérations : `RESOURCE`, `ACT_TYPE`, `ACT_INST`, et éventuellement `ROLE` ;
- des variables de décision : `start_time`, `assignment`, `role_assignment` ;
- des contraintes et une éventuelle fonction objectif `penalty`.

Les variables centrales sont :

```
array[ACT_INST] of var 1..TOTAL_SLOTS: start_time;
array[ACT_INST, RESOURCE] of var bool: assignment;
array[ACT_INST, ROLE, RESOURCE] of var bool: role_assignment;
```

Le sens attendu est le suivant :

- `start_time[a] = t` signifie que l'instance d'activité a commence au créneau global t ;
- `assignment[a,r] = true` signifie que la ressource r participe à l'activité a ;
- `role_assignment[a,ro,r] = true` signifie que la ressource r joue le rôle ro dans l'activité a .

6 Fonctions de correspondance

6.1 Fonction d'encodage

La fonction `encode` transforme une solution DSL en affectation de variables MINIZINC :

$$\text{encode} : \text{Sol}_{DSL}(P) \longrightarrow \text{Sol}_{MZN}(\text{Gen}(P))$$

Elle est définie ainsi :

- si une instance a commence au créneau t dans le DSL, alors `start_time[a] = t` ;
- si une ressource r participe à a , alors `assignment[a,r] = true` ;
- si r joue le rôle ro dans a , alors `role_assignment[a,ro,r] = true` ;
- toutes les autres affectations booléennes non utilisées valent `false`.

6.2 Fonction de décodage

La fonction `decode` transforme une sortie MINIZINC en solution DSL :

$$\text{decode} : \text{Sol}_{MZN}(\text{Gen}(P)) \longrightarrow \text{Sol}_{DSL}(P)$$

Dans l'implémentation, cette fonction est matérialisée par la sortie :

```
ACTIVITY: <instance> slot=<globalSlot> day=<day>
ASSIGNMENT: <instance> resource=<resource>
ROLE: <instance> role=<role> resource=<resource>
```

Puis par un parseur qui reconstruit, pour chaque instance, son jour, son créneau local, ses ressources participantes et ses rôles.

7 Correspondance élément par élément

Cette section montre comment chaque construction du DSL est traduite en MINIZINC en conservant son sens.

7.1 Temps

Construction DSL

Le DSL contient :

```
"time": {
  "days": ["Lundi", "Mardi"],
  "slotsPerDay": 4
}
```

Sa signification est : le planning est composé d'un nombre fini de jours, chaque jour contenant le même nombre de créneaux.

Traduction MiniZinc

Le générateur produit :

```
int: NUM_DAYS = ...;
int: SLOTS_PER_DAY = ...;
int: TOTAL_SLOTS = NUM_DAYS * SLOTS_PER_DAY;
function var int: dayOf(var int: t) = ((t-1) div SLOTS_PER_DAY) + 1;
function int: dayStart(int: d) = (d-1)*SLOTS_PER_DAY + 1;
function int: dayEnd(int: d) = d*SLOTS_PER_DAY;
```

Argument de conservation du sens

Le DSL définit les créneaux par paires (d, k) où d est un jour et k un créneau local du jour. Le modèle MINIZINC les représente par un créneau global :

$$t = (d - 1) \times SLOTS_PER_DAY + k.$$

La fonction `dayOf` permet de retrouver le jour à partir du créneau global. Ainsi, la représentation globale de MINIZINC est équivalente à la représentation jour/créneau du DSL.

Le générateur ajoute aussi :

```
constraint forall(i in ACT_INST) (
  ((start_time[i] - 1) mod SLOTS_PER_DAY) + duration[i] <= SLOTS_PER_DAY
);
```

Cette contrainte garantit qu'une activité ne déborde pas sur le jour suivant. Elle conserve donc le sens naturel d'une activité planifiée dans une journée.

Proposition 7.1 (Préservation du temps). *Toute valeur de début produite par MINIZINC correspond à un créneau global valide du DSL, et tout créneau valide du DSL correspond à une valeur possible de `start_time`, sous réserve des autres contraintes.*

Démonstration. Le domaine de `start_time` est $1..TOTAL_SLOTS$. Or $TOTAL_SLOTS = NUM_DAYS * SLOTS_PER_DAY$. Toute valeur de ce domaine appartient donc à l'ensemble des créneaux définis par le DSL. Réciproquement, tout couple (d, k) valide du DSL se traduit par un entier t compris entre 1 et $TOTAL_SLOTS$. La contrainte de non-débordement assure que la durée reste dans la journée de départ. $\square$

7.2 Activités

Construction DSL

Une activité est déclarée par un nom, un nombre d'instances et une durée optionnelle :

```
"activities": {
  "Soutenance": { "count": 3, "duration": 2 }
}
```

Si la durée n'est pas indiquée, l'implémentation lui donne la valeur par défaut 1.

Traduction MiniZinc

Le générateur produit :

```
enum ACT_TYPE = { Soutenance };
enum ACT_INST = { Soutenance_1, Soutenance_2, Soutenance_3 };
array[ACT_INST] of int: duration = [2, 2, 2];
array[ACT_INST] of ACT_TYPE: act_type = [Soutenance, Soutenance, Soutenance];
```

Argument de conservation du sens

Chaque activité déclarée avec `count = n` devient exactement n instances dans `ACT_INST`. La durée de chaque instance est copiée dans le tableau `duration`. Le tableau `act_type` conserve le lien entre l'instance et son type d'activité.

Proposition 7.2 (Préservation des activités). *Pour chaque activité A déclarée dans le DSL avec n instances et une durée d , le modèle MINIZINC contient exactement n instances de type A , chacune associée à la durée d .*

Démonstration. Le générateur parcourt chaque activité et crée les identifiants $A_1, \dots, A_n$. Pour chacun de ces identifiants, il ajoute une entrée dans **duration** et dans **act_type**. Il n'existe aucune autre source de création d'instances. La correspondance est donc bijective. $\square$

7.3 Ressources typées

Construction DSL

Le DSL classe les ressources par type :

```
"resources": {
  "Teacher": ["T1", "T2"],
  "Room": ["R1", "R2"]
}
```

Traduction MiniZinc

Le générateur produit une énumération globale et des ensembles typés :

```
enum RESOURCE = { T1, T2, R1, R2 };
set of RESOURCE: RES_Teacher = { T1, T2 };
set of RESOURCE: RES_Room = { R1, R2 };
```

Argument de conservation du sens

Le DSL distingue les ressources par type. MINIZINC représente toutes les ressources dans une énumération globale, puis reconstitue les types par des sous-ensembles. Une ressource appartient donc au même type dans MINIZINC que dans le DSL.

Proposition 7.3 (Préservation du typage des ressources). *Une ressource déclarée dans un type $RType$ du DSL appartient exactement à l'ensemble RES_RType correspondant dans MINIZINC.*

Démonstration. Le générateur parcourt chaque entrée **ResourceEntry**. Les instances sont ajoutées à **RESOURCE** et simultanément à l'ensemble **RES_Type**. Ainsi, l'appartenance typée du DSL est conservée dans MINIZINC. $\square$

7.4 Rôles

Construction DSL

Le DSL associe des rôles à chaque type d'activité :

```
"roles": {
  "Soutenance": {
    "President": "Teacher",
    "Salle": "Room"
  }
}
```

Cela signifie que les instances de **Soutenance** peuvent recevoir un rôle **President** joué par une ressource de type **Teacher**, et un rôle **Salle** joué par une ressource de type **Room**.

Traduction MiniZinc

Le générateur produit :

```
enum ROLE = { President, Salle };
array[ACT_INST, ROLE] of bool: role_applicable = ...;
array[ACT_INST, ROLE, RESOURCE] of var bool: role_assignment;
```

Puis il impose :

```
constraint forall(a in ACT_INST, ro in ROLE, r in RESOURCE) (
  if not role_applicable[a, ro]
  then role_assignment[a, ro, r] = false
  else true endif
);
```

et, pour chaque rôle, l'incompatibilité des types :

```
constraint forall(a in ACT_INST where act_type[a] == Soutenance,
  r in RESOURCE where not (r in RES_Teacher)) (
  role_assignment[a, President, r] = false
);
```

Enfin, toute ressource jouant un rôle est aussi marquée comme participante :

```
constraint forall(a in ACT_INST where act_type[a] == Soutenance,
  r in RES_Teacher) (
  role_assignment[a, President, r] -> assignment[a, r]
);
```

Argument de conservation du sens

La matrice `role_applicable` conserve la relation entre une activité et ses rôles. Les contraintes de compatibilité empêchent d'affecter une ressource d'un mauvais type à un rôle. L'implication vers `assignment` garantit qu'une ressource qui joue un rôle participe bien à l'activité.

Proposition 7.4 (Préservation des rôles). *Si le DSL déclare qu'un rôle `ro` d'une activité `A` attend une ressource de type `RType`, alors le modèle MINIZINC n'autorise pour ce rôle que les ressources de `RES_RType` et interdit ce rôle pour les autres activités.*

Démonstration. La construction `role_applicable` vaut vrai uniquement pour les couples activité-rôle déclarés dans le DSL. Lorsque le rôle n'est pas applicable, toutes les variables `role_assignment` correspondantes sont contraintes à `false`. Lorsque le rôle est applicable, les ressources hors du type attendu sont également contraintes à `false`. Les seules affectations possibles sont donc celles permises par le DSL. $\square$

7.5 Affectation générale des ressources

Le générateur utilise aussi une variable générale :

```
array[ACT_INST, RESOURCE] of var bool: assignment;
```

Cette variable représente la participation d'une ressource à une activité, indépendamment du rôle précis. Elle est reliée aux rôles par l'implication :

```
role_assignment[a, ro, r] -> assignment[a, r]
```

et par la contrainte :

```
constraint forall(a in ACT_INST, r in RESOURCE) (  
    sum(ro in ROLE where role_applicable[a, ro])  
        (bool2int(role_assignment[a, ro, r])) <= 1  
);
```

Cela signifie qu'une même ressource ne peut pas jouer deux rôles différents dans la même instance d'activité.

8 Préservation des contraintes dures

Les contraintes dures définissent l'espace des solutions admissibles. Pour chacune, nous montrons que la traduction MINIZINC exprime la même condition.

8.1 Contrainte mandatory_roles

Sens DSL

```
{ "type": "mandatory_roles", "activity": "Soutenance" }
```

Tous les rôles applicables à chaque instance de l'activité concernée doivent recevoir au moins une ressource.

Traduction MiniZinc

```
constraint forall(a in ACT_INST where act_type[a] == Soutenance,  
    ro in ROLE where role_applicable[a, ro]) (  
    sum(r in RESOURCE) (bool2int(role_assignment[a, ro, r])) >= 1  
);
```

Preuve

La somme compte le nombre de ressources affectées au rôle `ro` dans l'instance `a`. La contrainte impose que cette somme soit au moins égale à 1. C'est exactement le sens de l'obligation de remplir chaque rôle applicable.

8.2 Contrainte cardinality_per_activity

Cas 1 : cardinalité sur un type de ressource

DSL :

```
{  
    "type": "cardinality_per_activity",  
    "activity": "Soutenance",  
    "target": "Teacher",  
    "min": 2,  
}
```

```
"max": 3
}
```

Traduction MINIZINC :

```
constraint forall(a in ACT_INST where a in { Soutenance_1, Soutenance_2 }) (
    sum(r in RES_Teacher) (bool2int(assignment[a,r])) >= 2 /\
    sum(r in RES_Teacher) (bool2int(assignment[a,r])) <= 3
);
```

La somme compte les ressources du type ciblé affectées à chaque instance d'activité. La borne inférieure et la borne supérieure sont recopiées depuis le DSL. Le sens est donc préservé.

Cas 2 : cardinalité sur un rôle

DSL :

```
{
  "type": "cardinality_per_activity",
  "activity": "Soutenance",
  "role": "President",
  "min": 1,
  "max": 1
}
```

Traduction MINIZINC :

```
constraint forall(a in ACT_INST where a in { Soutenance_1, Soutenance_2 }) (
    sum(r in RES_Teacher)
      (bool2int(role_assignment[a, President, r])) >= 1 /\
    sum(r in RES_Teacher)
      (bool2int(role_assignment[a, President, r])) <= 1
);
```

La somme compte les ressources affectées au rôle. Les bornes imposent exactement la cardinalité demandée.

Cas 3 : cardinalité sur slot

Dans l'implémentation actuelle, chaque instance d'activité possède exactement une variable `start_time`. Le générateur traduit donc une cardinalité sur `slot` par :

```
constraint min <= 1 /\ 1 <= max;
```

Cette traduction exprime que l'activité possède toujours exactement un créneau de début.

8.3 Contrainte `resource_exclusivity`

Sens DSL

Cette contrainte limite le nombre d'utilisations d'une ressource d'un type donné, pour une activité donnée, selon un périmètre `slot` ou `day`.

Exemple :

```
{
  "type": "resource_exclusivity",
  "resourceType": "Teacher",
  "activity": "Soutenance",
  "scope": "slot",
  "max": 1
}
```

Traduction MiniZinc pour slot

```
constraint forall(r in RES_Teacher, t in 1..TOTAL_SLOTS) (
  sum(a in ACT_INST where act_type[a] == Soutenance) (
    bool2int(assignment[a,r] /\
      start_time[a] <= t /\
      t < start_time[a] + duration[a])
  ) <= 1
);
```

Cette contrainte compte, pour chaque ressource et chaque créneau, le nombre d'activités qui utilisent cette ressource à ce moment. La valeur ne doit pas dépasser `max`.

Traduction MiniZinc pour day

```
constraint forall(r in RES_Teacher, d in 1..NUM_DAYS) (
  sum(a in ACT_INST where act_type[a] == Soutenance) (
    bool2int(assignment[a,r] /\ dayOf(start_time[a]) == d)
  ) <= 1
);
```

Cette variante compte les participations par jour.

Preuve

Dans le cas `slot`, la condition

$$start_time[a] \leq t < start_time[a] + duration[a]$$

signifie exactement que l'activité a occupe le créneau t . La somme représente donc le nombre d'activités utilisant la ressource r au créneau t . La borne `max` est celle du DSL. Dans le cas `day`, la fonction `dayOf` ramène le créneau global au jour. La contrainte est donc équivalente à la limite journalière du DSL.

8.4 Non-chevauchement global des ressources

En plus des contraintes explicites du DSL, le générateur ajoute une règle générique forte :

```
constraint forall(r in RESOURCE) (
  forall(i, j in ACT_INST where i < j) (
    (assignment[i, r] /\ assignment[j, r]) ->
```

```

    (start_time[i] + duration[i] <= start_time[j] \/  

     start_time[j] + duration[j] <= start_time[i])  

  )  

);

```

Cette contrainte exprime qu’une même ressource ne peut pas être utilisée simultanément par deux activités quelconques.

Remarque 8.1. *Cette règle est plus générale que la contrainte `resource_exclusivity` ciblée par type et par activité. Pour que la bisimulation soit exacte, cette règle doit être considérée comme faisant partie de la sémantique opérationnelle du DSL dans l’implémentation : toute ressource participante est indisponible pendant la durée de l’activité. Si l’on voulait une sémantique DSL plus permissive, il faudrait rendre cette règle optionnelle ou exclusivement pilotée par les contraintes explicites.*

8.5 Contrainte `fixed_assignment`

Sens DSL

Une ressource doit obligatoirement être affectée à un rôle donné d’une instance d’activité.

```

{  
  "type": "fixed_assignment",  
  "activityInstance": "Soutenance_1",  
  "role": "President",  
  "resource": "T1"  
}
```

Traduction MiniZinc

```

constraint assignment[Soutenance_1, T1] = true;  
constraint role_assignment[Soutenance_1, President, T1] = true;
```

Preuve

La première contrainte impose la participation de la ressource à l’activité. La seconde impose son rôle précis. Toute solution MINIZINC contient donc l’affectation demandée, et toute solution DSL valide doit la contenir.

8.6 Contrainte `forbidden_assignment`

Sens DSL

Une ressource ne doit pas être affectée à un rôle donné d’une instance d’activité.

Traduction MiniZinc

```

constraint role_assignment[Soutenance_1, President, T1] = false;
```

Preuve

La variable booléenne représentant exactement cette affectation est contrainte à **false**. La traduction interdit donc précisément l'affectation interdite par le DSL.

8.7 Contrainte temporal_precedence

Sens DSL

Une activité de type *A* doit se terminer avant une activité de type *B*.

```
{
  "type": "temporal_precedence",
  "beforeActivity": "Preparation",
  "afterActivity": "Soutenance"
}
```

Traduction MiniZinc

Le générateur impose cette contrainte pour toutes les paires d'instances concernées :

```
constraint forall(b in { Preparation_1, Preparation_2 },
                 a in { Soutenance_1, Soutenance_2 }) (
  start_time[b] + duration[b] <= start_time[a]
);
```

Preuve

L'expression `start_time[b] + duration[b]` représente le premier créneau après la fin de *b*. L'inégalité impose que *a* commence après cette fin. Comme la contrainte est appliquée à toutes les instances de l'activité avant et de l'activité après, elle exprime exactement la précedence globale du DSL.

8.8 Contrainte instance_precedence

Sens DSL

Une instance particulière doit précéder une autre instance particulière.

Traduction MiniZinc

```
constraint start_time[Preparation_1] + duration[Preparation_1]
  <= start_time[Soutenance_1];
```

Preuve

C'est la même condition temporelle que précédemment, mais appliquée à deux instances nommées. La traduction conserve donc exactement le sens de la contrainte.

8.9 Contrainte `time_window`

Sens DSL

Une instance d'activité doit être planifiée entre deux créneaux globaux.

```
{  
  "type": "time_window",  
  "activityInstance": "Soutenance_1",  
  "minSlot": 3,  
  "maxSlot": 8  
}
```

Traduction MiniZinc

```
constraint start_time[Soutenance_1] >= 3;  
constraint start_time[Soutenance_1] + duration[Soutenance_1] - 1 <= 8;
```

Preuve

La première contrainte impose que l'activité ne commence pas avant `minSlot`. La seconde impose que son dernier créneau occupé ne dépasse pas `maxSlot`. Cela correspond exactement au sens d'une fenêtre temporelle fermée.

8.10 Contrainte `required_resource`

Sens DSL

Une instance d'activité requiert la participation d'une ressource donnée, sans nécessairement préciser son rôle.

Traduction MiniZinc

```
constraint assignment[Soutenance_1, T1] = true;
```

Preuve

La variable de participation de la ressource dans l'activité est contrainte à vrai. La ressource est donc nécessairement présente dans toute solution MINIZINC, comme demandé dans le DSL.

9 Préservation des préférences

Les préférences ne définissent pas l'espace des solutions valides, mais la qualité des solutions. Le générateur les traduit en termes de pénalité. Si au moins une préférence existe, le modèle contient :

```

var int: penalty = ...;
solve :: int_search([start_time[i] | i in ACT_INST],
                    first_fail, indomain_min, complete)
    minimize penalty;

```

Sinon, il résout simplement un problème de satisfaction :

```

solve :: int_search([start_time[i] | i in ACT_INST],
                    first_fail, indomain_min, complete)
    satisfy;

```

La propriété attendue est :

$$\text{score}_{DSL}(s) = \text{score}_{MZN}(\text{encode}(s)).$$

9.1 Préférence avoid_participation_on_date

Sens DSL

Une ressource devrait éviter de participer à une activité à une date donnée. Si elle participe au moins une fois ce jour-là, une pénalité est ajoutée.

Traduction MiniZinc

```

weight * bool2int(
    sum(a in ACT_INST)
        (bool2int(assignment[a, Ressource] /\ dayOf(start_time[a]) == d)) > 0
)

```

Argument

La somme détecte les activités du jour d auxquelles la ressource participe. Le booléen vaut vrai si le nombre de participations est strictement positif. La pénalité est donc appliquée une seule fois, conformément à une interprétation forfaitaire de cette préférence.

9.2 Préférence preferred_resource

Sens DSL

Une ressource est préférée pour un rôle ou une activité. Ne pas choisir cette ressource ajoute une pénalité.

Traduction MiniZinc

Avec rôles :

```

weight * bool2int(not role_assignment[Instance, Role, Resource])

```

Sans rôles :

```

weight * bool2int(not assignment[Instance, Resource])

```


Argument

La pénalité est nulle si la ressource préférée est effectivement choisie. Elle vaut `weight` sinon. Cela correspond exactement à la préférence du DSL.

9.3 Préférence `max_per_scope`

Sens DSL

On souhaite limiter souplement le nombre de participations d'une ressource, par créneau ou par jour. Dépasser la limite n'invalide pas la solution, mais ajoute une pénalité proportionnelle à l'excès.

Traduction MiniZinc pour `slot`

```
sum(r in RES_Type, t in 1..TOTAL_SLOTS) (  
  weight * max(0,  
    (sum(a in ACT_INST where act_type[a] == Activity)  
      (bool2int(assignment[a, r] /\  
        start_time[a] <= t /\  
        t < start_time[a] + duration[a]))) - max  
  )  
)
```

Argument

La formule compte les utilisations effectives de la ressource dans chaque périmètre. La fonction `max(0, count - max)` calcule uniquement le dépassement positif. Le poids du DSL multiplie ce dépassement. La pénalité correspond donc à l'interprétation souple de la limite.

9.4 Préférence `room_stability_for_role`

Sens DSL

Pour une activité et un rôle donnés, on souhaite qu'un même acteur conserve la même salle, soit sur une journée, soit globalement.

Traduction MiniZinc

Le générateur compte, pour chaque acteur jouant le rôle, le nombre de salles différentes utilisées. Il pénalise le nombre de salles au-delà de 1 :

```
weight * max(0, number_of_used_rooms - 1)
```

Argument

Si l'acteur utilise une seule salle, la pénalité est nulle. Chaque salle supplémentaire crée une pénalité pondérée. Cela correspond au sens de stabilité de salle.

9.5 Préférence compact_schedule_for_role

Sens DSL

Pour une activité et un rôle donnés, on souhaite que l'emploi du temps d'une ressource soit compact, c'est-à-dire avec peu de trous entre ses participations.

Traduction MiniZinc

Le générateur calcule :

- `busy_units` : le nombre total de créneaux réellement occupés ;
- `first_slot` : le premier créneau occupé ;
- `last_slot` : le dernier créneau occupé.

La pénalité est :

$$weight \times ((last_slot - first_slot + 1) - busy_units).$$

Argument

Le terme $(last_slot - first_slot + 1)$ mesure la longueur de l'enveloppe temporelle. En retirant les créneaux réellement occupés, on obtient le nombre de trous. La pénalité représente donc exactement le manque de compacité.

10 Théorème principal de bisimulation

Théorème 10.1 (Correction et complétude de la traduction). *Soit P une spécification PLANNINGSPEC bien formée, et soit $Gen(P)$ le modèle MINIZINC généré. Sous l'hypothèse que toutes les références du DSL sont valides et que seuls les périmètres supportés par l'implémentation sont utilisés, alors pour toute solution s :*

$$s \models P \iff encode(s) \models Gen(P).$$

De plus, pour toute solution MINIZINC m :

$$m \models Gen(P) \implies decode(m) \models P.$$

Démonstration. La preuve se fait par construction sur les éléments de la spécification.

Premièrement, les éléments structurels sont conservés. Les jours et créneaux du DSL sont représentés par `NUM_DAYS`, `SLOTS_PER_DAY` et `TOTAL_SLOTS`. Les activités du DSL sont représentées par `ACT_TYPE` et `ACT_INST`. Les ressources typées sont représentées par `RESOURCE` et les ensembles `RES_Type`. Les rôles sont représentés par `ROLE`, `role_applicable` et `role_assignment`.

Deuxièmement, les variables de décision MINIZINC correspondent exactement aux choix constitutifs d'une solution DSL : date de début, participation d'une ressource et affectation d'un rôle. La fonction `encode` affecte ces variables selon les choix de la solution DSL. La fonction `decode` lit ces mêmes variables dans la sortie structurée.

Troisièmement, chaque contrainte dure du DSL est traduite par une formule MINIZINC équivalente : les cardinalités sont traduites par des sommes bornées, les précédences par des inégalités sur les dates de fin et de début, les fenêtres temporelles par des bornes sur

le début et la fin, les affectations fixes ou interdites par des égalités booléennes, et les ressources obligatoires par des variables de participation contraintes à vrai.

Ainsi, si une solution DSL satisfait toutes les contraintes du DSL, son encodage satisfait toutes les contraintes MINIZINC. Réciproquement, si une affectation MINIZINC satisfait toutes les contraintes générées, son décodage respecte les contraintes du DSL, puisque chaque contrainte MINIZINC exprime précisément la condition DSL correspondante. Les deux ensembles de solutions sont donc équivalents, à encodage et décodage près. $\square$

Théorème 10.2 (Préservation de l’optimisation). *Si une spécification P contient des préférences et si $\text{Gen}(P)$ génère la variable **penalty**, alors pour toute solution DSL s :*

$$\text{score}_{DSL}(s) = \text{score}_{MZN}(\text{encode}(s)).$$

*Par conséquent, toute solution optimale minimisant **penalty** dans MINIZINC correspond à une solution optimale du DSL selon les préférences implémentées.*

Démonstration. Chaque préférence du DSL ajoute un terme à **penalty**. Le terme produit par le générateur est nul lorsque la préférence est satisfaite et strictement positif, selon le poids fourni par le DSL, lorsqu’elle est violée. Pour **preferred_resource**, la pénalité est exactement le poids si la ressource préférée n’est pas choisie. Pour **avoid_participation_on_date**, elle est appliquée si la ressource participe au moins une fois à la date indiquée. Pour **max_per_scope**, elle correspond au dépassement de la limite. Pour **room_stability_for_role**, elle correspond au nombre de salles supplémentaires. Pour **compact_schedule_for_role**, elle correspond au nombre de trous dans l’enveloppe temporelle. La somme de ces termes est donc égale au score défini par la sémantique opérationnelle du DSL. $\square$

11 Table de synthèse de la correspondance

TABLE 1: Synthèse de la correspondance sémantique entre PLANNINGSPEC et MINIZINC.

Élément DSL	Traduction MiniZinc	Sens conservé
<code>time.days</code>	<code>NUM_DAYS</code> , fonction <code>dayOf</code>	Les jours du DSL sont retrouvables à partir des créneaux globaux.
<code>slotsPerDay</code>	<code>SLOTS_PER_DAY</code> , <code>TOTAL_SLOTS</code>	Le nombre total de créneaux est identique.
<code>activities</code>	<code>ACT_TYPE</code> , <code>ACT_INST</code>	Chaque activité et chaque instance sont conservées.
<code>duration</code>	tableau <code>duration</code>	La durée utilisée par le solveur est celle du DSL.
<code>resources</code>	<code>RESOURCE</code> , <code>RES_Type</code>	Le typage des ressources est conservé.
<code>roles</code>	<code>ROLE</code> , <code>role_applicable</code> , <code>role_assignment</code>	Les rôles applicables et leurs types de ressources sont conservés.

Élément DSL	Traduction MiniZinc	Sens conservé
<code>mandatory_roles</code>	somme des <code>role_assignment</code> ≥ 1	Chaque rôle obligatoire reçoit une ressource.
<code>cardinality_per_activity</code>	sommes bornées par <code>min</code> et <code>max</code>	Les nombres de ressources ou rôles affectés respectent les bornes.
<code>resource_exclusivity</code>	sommes par créneau ou par jour $\leq max$	Les limites d'utilisation des ressources sont conservées.
<code>fixed_assignment</code>	égalité booléenne à <code>true</code>	L'affectation imposée est présente.
<code>forbidden_assignment</code>	égalité booléenne à <code>false</code>	L'affectation interdite est absente.
<code>temporal_precedence</code>	<code>start_before + duration <= start_after</code>	La précédence entre types d'activités est conservée.
<code>instance_precedence</code>	même inégalité entre instances	La précédence entre instances est conservée.
<code>time_window</code>	bornes sur début et fin	L'activité reste dans la fenêtre autorisée.
<code>required_resource</code>	<code>assignment[a,r] = true</code>	La ressource requise participe à l'activité.
<code>preferences</code>	variable <code>penalty</code>	Les violations souples sont évaluées par le même score.

12 Hypothèses et limites de la preuve

La preuve précédente s'appuie sur les hypothèses suivantes :

1. La spécification DSL est syntaxiquement valide selon la grammaire Langium.
2. Les références utilisées dans les contraintes existent effectivement : activité, instance, rôle, ressource et type de ressource.
3. Les périmètres utilisés sont ceux supportés par l'implémentation : `slot` et `day` pour `resource_exclusivity` et `max_per_scope`, `day` et `global` pour `room_stability_for_role` et `compact_schedule_for_role`.
4. La règle de non-chevauchement global des ressources est considérée comme faisant partie de la sémantique du DSL implémenté.
5. La normalisation des identifiants du DSL vers MINIZINC ne provoque pas de collision de noms. Par exemple, deux noms différents du DSL ne doivent pas devenir le même identifiant MINIZINC après remplacement des espaces et caractères spéciaux.

Ces hypothèses sont importantes. Elles ne remettent pas en cause la traduction, mais elles indiquent les conditions sous lesquelles l'équivalence sémantique est garantie. Pour renforcer encore le projet, il serait utile d'ajouter des validations statiques supplémentaires détectant les références inconnues et les collisions d'identifiants.

13 Conclusion

Cette étude montre que la traduction de PLANNINGSPEC vers MINIZINC peut être comprise comme une bisimulation sémantique adaptée aux langages déclaratifs. Le DSL exprime un problème de planification à un niveau métier ; MINIZINC exprime le même problème sous forme de variables, domaines, contraintes et objectif. La correspondance repose sur deux fonctions conceptuelles : **encode**, qui transforme une solution DSL en affectation MINIZINC, et **decode**, qui reconstruit une solution DSL à partir de la sortie MINIZINC.

La preuve se fait élément par élément. Les jours, créneaux, activités, ressources, rôles, contraintes dures et préférences sont traduits par des constructions MINIZINC ayant le même sens. Les contraintes dures préservent l'espace des solutions valides. Les préférences préservent le score optimisé par le solveur.

La propriété centrale obtenue est donc :

$$s \models P \iff \text{encode}(s) \models \text{Gen}(P),$$

et, pour l'optimisation :

$$\text{score}_{DSL}(s) = \text{score}_{MZN}(\text{encode}(s)).$$

Ainsi, sous les hypothèses explicitées, MiniZinc ne résout pas un problème différent de celui exprimé par l'utilisateur : il résout exactement la traduction sémantiquement équivalente de la spécification PLANNINGSPEC.